Transformer Parameter Calculations

what are the calculations for the parameter count of a standard transformer model in pytorch? i need to tweak it to match a previous model

For a standard PyTorch Transformer, parameter count is primarily determined by four things:

D-OD-1

- `d_model`: hidden/embedding dimension
- `dim_feedforward`: feed-forward network width
- `num_encoder_layers` and `num_decoder_layers`: Transformer depth
- Whether you include token embeddings, output projections, biases, and final LayerNorms

The interesting part is that `nhead` **does not directly change parameter count**, provided `d_model` stays fixed. It changes how the attention dimensions are partitioned, not the total size of the projection matrices.

D-DJ-2

1. Parameter count formulas

Let:

Symbol	PyTorch argument
d	`d_model`
f	`dim_feedforward`
L_e	`num_encoder_layers`
L_d	`num_decoder_layers`
V	Vocabulary size, if applicable

Assuming standard dense attention, enabled biases, and LayerNorm affine parameters:

Encoder layer

Each encoder layer contains self-attention, a feed-forward network, and two LayerNorms.

Component	Parameters
Q, K, V projection weights	$3d^2$
Attention output projection	d^2
Attention biases	$4d$
Feed-forward weights	$2df$
Feed-forward biases	$f + d$
Two LayerNorms	$4d$

Therefore:

$$\boxed{P_{\text{encoder layer}} = 4d^2 + 2df + 9d + f}$$

Decoder layer

A standard encoder-decoder Transformer decoder has both causal self-attention and encoder-decoder cross-attention.

$$\boxed{P_{\text{decoder layer}} = 8d^2 + 2df + 15d + f}$$

The decoder has twice the attention projections and three LayerNorms.

Complete Transformer

For PyTorch's `nn.Transformer` with its usual final encoder and decoder LayerNorms:

$$\boxed{P_{\text{total}} = L_e(4d^2 + 2df + 9d + f) + L_d(8d^2 + 2df + 15d + f) + 4d}$$

This excludes token embeddings and vocabulary output heads, which `nn.Transformer` does not instantiate.

2. Verified PyTorch example

For the default `nn.Transformer` configuration:

```
import torch.nn as nn

model = nn.Transformer(
    d_model=512,
    nhead=8,
    num_encoder_layers=6,
    num_decoder_layers=6,
    dim_feedforward=2048,
)

params = sum(p.numel() for p in model.parameters())
print(f"{params:,}")
```

TOTAL PARAMETERS

44,140,544

Verified against PyTorch 2.10.0

Component	Parameter count
6 encoder layers	18,914,304
6 decoder layers	25,224,192
Final LayerNorms	2,048
Total	**44,140,544**

This matches the current PyTorch implementation, including the two final LayerNorms.

3. Matching an existing parameter budget

If you already know the previous model's parameter count, you can solve directly for `dim_feedforward` while keeping `d_model` and the number of layers fixed.

Rearranging the full Transformer equation:

$$\boxed{f = \frac{P_{\text{target}} - (4L_e + 8L_d)d^2 - (9L_e + 15L_d + 4)d}{(2d + 1)(L_e + L_d)}}$$

Round the result to a positive integer to obtain an allowable feed-forward dimension. Exact matching may not be possible with this parameter alone.

Here is a reusable implementation:

```
import torch.nn as nn

def transformer_params(d, f, enc_layers, dec_layers):
    encoder = 4*d**2 + 2*d*f + 9*d + f
    decoder = 8*d**2 + 2*d*f + 15*d + f
    return enc_layers*encoder + dec_layers*decoder + 4*d

def match_ff_dim(target, d, enc_layers, dec_layers):
    base = (
        (4*enc_layers + 8*dec_layers)*d**2
        + (9*enc_layers + 15*dec_layers + 4)*d
    )
    coefficient = (2*d + 1)*(enc_layers + dec_layers)
    if coefficient == 0:
        raise ValueError("At least one layer is required")
    return max(1, round((target - base) / coefficient))

target = 44_000_000
d = 512
le = 6
ld = 6

f = match_ff_dim(target, d, le, ld)
actual = transformer_params(d, f, le, ld)

print("Suggested dim_feedforward:", f)
print("Actual parameters:", actual)
print("Difference:", actual - target)
```

4. Which knobs change the parameter count?

Change	Effect
Increase `d_model`	Approximately quadratic growth
Increase `dim_feedforward`	Linear growth
Add encoder/decoder layers	Linear growth
Increase `nhead`, fixed `d_model`	No change
Change `dropout`	No change
Change sequence length	No change unless positional embeddings are learned
Disable biases	Small reduction

Important distinction: if your previous model was GPT-style (decoder-only with causal self-attention), do **not** use the `TransformerDecoderLayer` formula above. PyTorch's decoder layer includes cross-attention, whereas a standard GPT block does not.

For a GPT-style block with ordinary two-layer MLP and LayerNorm:

$$P_{\text{GPT block}} = 4d^2 + 2df + 9d + f$$

It's the same parameter count as the encoder-layer formula, despite using causal masking.

Then add:

$$P_{\text{GPT total}} = L(4d^2 + 2df + 9d + f) + P_{\text{embeddings}} + P_{\text{head}} + 2d$$

For example, token embeddings add Vd, and learned positional embeddings add Td. A tied output projection reuses the token embedding matrix and should not be counted twice.

What was your previous model's architecture and parameter count? If you give me its `d_model`, number of layers, `dim_feedforward`, vocabulary size, and whether it's encoder-only, decoder-only, or encoder-decoder, I can work out a configuration that gets as close as possible to the exact original parameter count.

why does a transformer do so much better than an lstm on the parity problem?

The interesting thing about the **parity problem** is that, theoretically, an LSTM should be very good at it. Parity requires only one bit of memory.

Yet in practice, Transformers can substantially outperform LSTMs on parity classification, especially during training.

The main distinction is:

An LSTM has to learn to maintain and update a parity state sequentially, whereas a Transformer can use attention and parallel representations to learn a different computation of parity. W-CV-3

But there's a subtle catch: **being better at fitting parity examples doesn't necessarily mean the Transformer has learned the underlying XOR algorithm.**

1. Why parity is deceptively difficult

Given a binary sequence:

$$x = [1, 0, 1, 1, 0, 1]$$

The parity is:

$$y = x_1 \oplus x_2 \oplus \cdots \oplus x_n$$

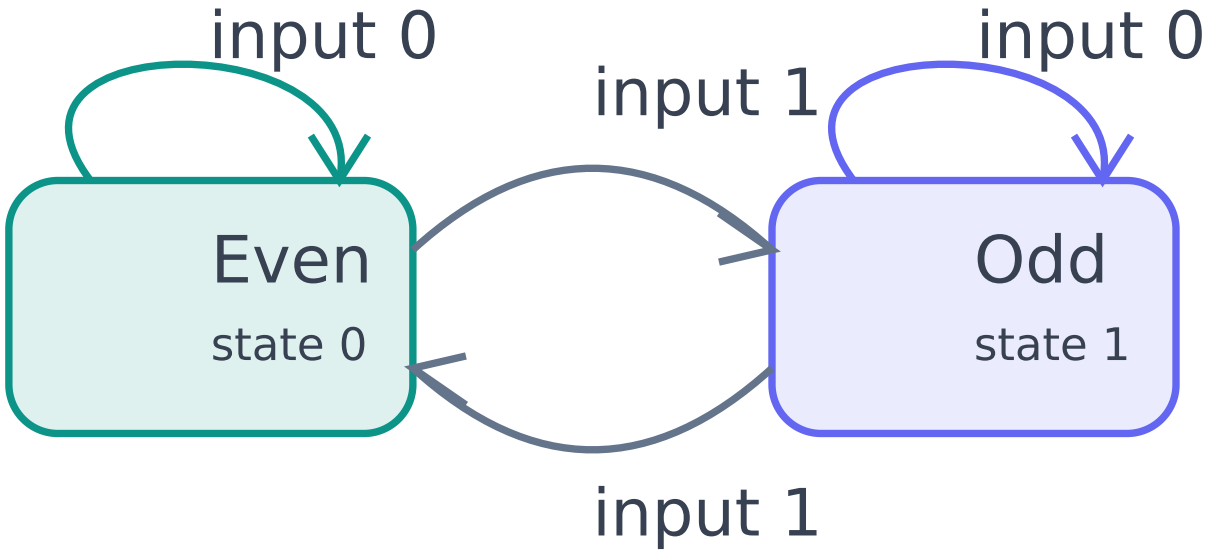
Here there are four ones, so the result is 0 (even parity).

An ideal recurrent model would maintain a state:

$$h_t = h_{t-1} \oplus x_t$$

This requires only two possible states: even or odd.

Ideal parity state machine



Reading 0 preserves the current state; reading 1 toggles it. This machine works for sequences of arbitrary length.

So the problem isn't memory capacity. The challenge is **learning a state transition that must remain accurate over many sequential steps**.

2. Why a Transformer might train better

A. Much shorter gradient paths

In an LSTM, information from the first input must propagate through every subsequent recurrent update.

Approximately:

$$\frac{\partial \mathcal{L}}{\partial h_1} = \frac{\partial \mathcal{L}}{\partial h_T} \prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}}$$

The product of these Jacobians can shrink, grow, or distort the gradient.

LSTM gates mitigate this, but parity requires something more difficult than simply remembering a value: the network must **toggle its representation every time it encounters a 1**.

A Transformer, by comparison, allows a final representation to depend directly on every input through self-attention. This can make fitting fixed-length examples easier because it avoids the same chain of recurrent derivatives.

However, shorter gradient paths alone do not solve parity's fundamental optimization difficulty.

B. The Transformer may learn counting instead of XOR

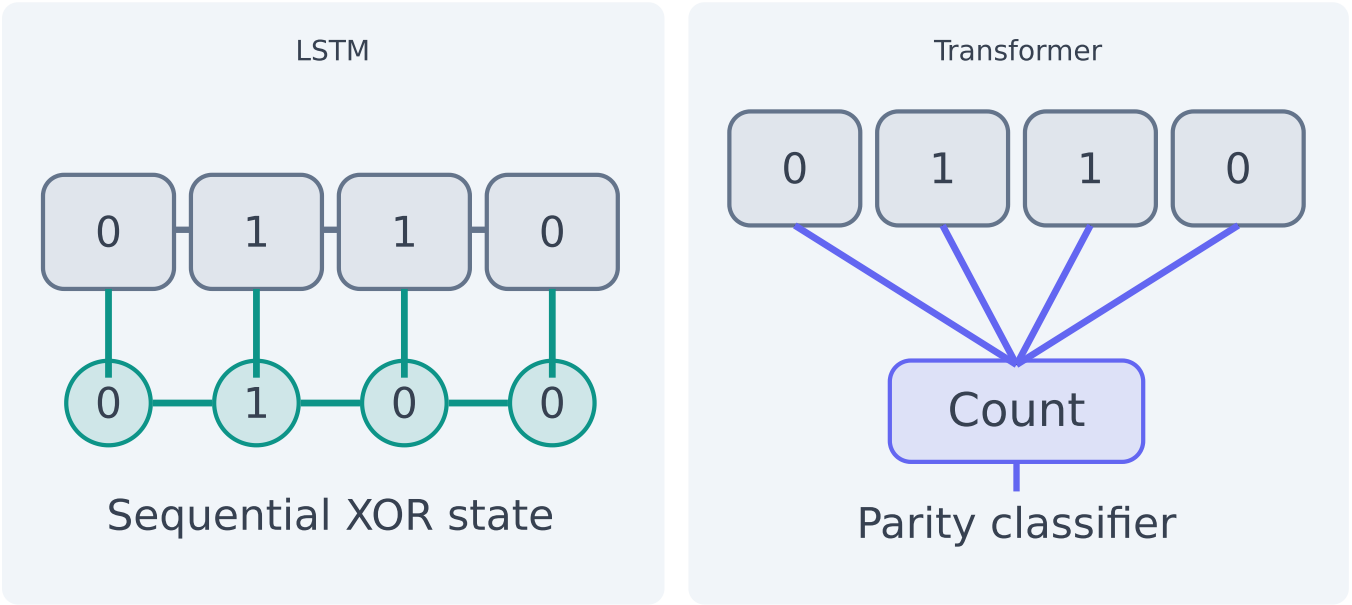
Suppose you always train on sequences of length 16.

Parity can be rewritten as:

$$y = \left(\sum_{i=1}^{16} x_i \right) \bmod 2$$

A Transformer with a pooled representation can potentially learn an approximate count of the ones, then use its feed-forward layers to classify counts as even or odd.

Two possible computational strategies



Conceptual illustration, not an assertion about the internal representations learned by your particular models.

The counting strategy could work exceptionally well over a fixed, limited range of lengths.

But it is not automatically a general XOR algorithm.

For example, the sequences with 2 ones out of 4 and 3 ones out of 6 both have a mean of 0.5, but opposite parity. A model that only knows the mean cannot distinguish them.

3. The deeper mathematical issue: parity has no lower-order signal

This is arguably the most fascinating property of parity.

Using inputs $x_i \in \{0, 1\}$, define:

$$g(x) = (-1)^{\sum_i x_i} = \prod_i (1 - 2x_i)$$

Under independent, uniformly sampled bits, parity is statistically independent of every proper subset of the input bits.

For example, knowing 15 out of 16 bits gives you absolutely no information about the answer if the sixteenth bit is random.

This makes parity a high-degree interaction problem. Learning useful local features does not necessarily provide a gradual path toward the correct solution.

Both Transformers and LSTMs can struggle with this. In fact, published experiments have found LSTMs outperforming Transformers on modular-counting and regular-language tasks.

This is why I would not attribute your result simply to attention being inherently better at parity.

4. How to determine what your Transformer actually learned

The most informative experiment is to test lengths beyond the training distribution.

Suppose you trained with length 16.

Test	What it reveals
New random sequences, length 16	Generalization within the training length
Sequences of length 24	Moderate length extrapolation
Sequences of length 32	Stronger length extrapolation
Flip one input bit at every position	Whether every bit influences the answer correctly
Compare equal numbers of ones at different lengths	Whether the model relies on count, fraction, or position

Here's a useful bit-flip test:

```
import torch

@torch.no_grad()
def parity_flip_test(model, x, predict):
    """
```

```
    x: binary tensor of shape [batch, seq_len]
    predict(model, x): predicted 0/1 labels [batch]
    """
    model.eval()

    original = predict(model, x).long()
    results = []

    for pos in range(x.shape[1]):
        flipped = x.clone()
        flipped[:, pos] = 1 - flipped[:, pos]

        new_pred = predict(model, flipped).long()

        # Correct parity must flip whenever one bit flips.
        flip_rate = (new_pred != original).float().mean()
        results.append(flip_rate.item())

    return results
```

The ideal flip rate is 1.0 at every position. This is a necessary test of parity behavior, though not sufficient proof that the model computes parity correctly.

Research has also shown that adding intermediate parity states (a scratchpad or chain of thought) can dramatically improve Transformers' ability to learn and generalize this algorithm.

5. What I'd investigate in your PyTorch experiment

Since you're also matching parameter counts between architectures, I would specifically check:

- **Parameter matching:** Equal parameter counts do not imply equal optimization difficulty. Transformers and LSTMs use their parameters differently.
- **Sequence readout:** An LSTM using its final hidden state has to preserve the entire running parity. A Transformer with global attention or pooling receives information from all positions.
- **Training and evaluation lengths:** Fixed-length experiments may reward a learned count-to-label mapping rather than a length-independent parity algorithm.

- **Optimization:** Learning rate, initialization, gate biases, gradient clipping, normalization, and dropout can significantly affect the LSTM result.
- **Hidden-state handling:** Verify that each independent training sequence starts with the intended LSTM hidden and cell states.

My key question is: **What sequence lengths are you training on, and what accuracies are you seeing for the Transformer versus the LSTM?**

For instance, a Transformer reaching 99% versus an LSTM stuck at 50% on length-16 sequences suggests a very different mechanism from a Transformer reaching 99% on length 100 and generalizing to length 1,000. The latter would be considerably more surprising.

it seems to converge much faster at about the same amount of parameters

That makes sense, and **converging much faster at roughly the same parameter count is a particularly interesting result**. It suggests the Transformer has a significant optimization advantage on your specific parity task, not necessarily greater representational capacity.

I suspect three mechanisms are responsible.

1. The LSTM must learn a precise recurrent operation

An LSTM updates its cell state using:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

For parity, the ideal operation is:

$$c_t = \begin{cases} c_{t-1}, & x_t = 0 \\ 1 - c_{t-1}, & x_t = 1 \end{cases}$$

That's harder than ordinary memory retention.

The LSTM has to learn gates that implement a conditional state toggle. Its recurrent weights must work correctly for every input position, regardless of the current state.

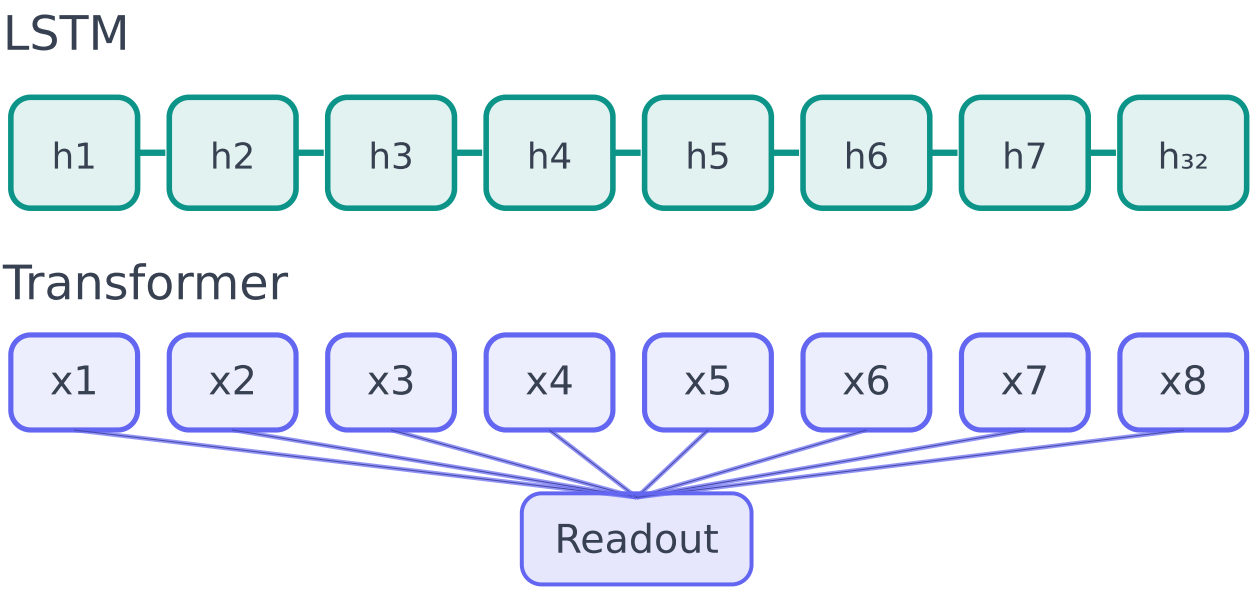
Early in training, the LSTM's hidden representations may not resemble parity states at all. Errors in the early recurrent transitions affect the representations that every subsequent transition receives.

The Transformer doesn't need to discover this particular recurrent state machine.

2. The Transformer has a more direct optimization path

Consider a length-32 sequence.

Computational dependency paths



Schematic: the LSTM has sequential state dependencies; a Transformer can aggregate information from all tokens through attention.

For an LSTM, gradients flow through a chain of recurrent updates. For a Transformer, every token can contribute to a global representation through a much shorter path.

This matters especially for parity because **every input bit must influence the answer**, including the first bit.

Also, Transformer token computations are highly parallelizable on GPUs. So if you're measuring wall-clock convergence, the Transformer may have both an optimization advantage and a hardware-throughput advantage.

3. The Transformer might find a simpler solution for fixed-length parity

This is the mechanism I'd be most interested in testing.

For sequences of fixed length T, a Transformer with global attention or pooling could potentially learn a representation of

$$s = \frac{1}{T} \sum_{t=1}^{T} x_t$$

Then a feed-forward network maps this statistic to the parity label.

Because T is fixed, the number of ones is recoverable from s. This converts parity into classification over $T + 1$ possible counts.

It's still a nontrivial alternating-label problem, but it's a completely different parameterization from learning XOR as a recurrent state transition.

A sufficiently expressive feed-forward network may fit that fixed range of counts more easily than an LSTM learns reliable sequential toggling.

The drawback: a model that fits this mapping might not generalize well to longer sequences.

4. A quick experiment to isolate the mechanism

I'd compare three quantities rather than just training accuracy.

Metric	Interpretation
Steps to 95% accuracy	Optimization efficiency
Seconds to 95% accuracy	Optimization plus hardware efficiency
Accuracy at 2× training length	Length generalization

And I'd test three models:

1. **LSTM** with final hidden-state classification.
2. **Transformer** with global token readout.
3. **MLP on the number of ones**, as a counting baseline.

The third model is surprisingly informative. If it also converges extremely quickly, that suggests much of the Transformer's advantage may be its ability to exploit a global

count-like representation.

One caveat: identical parameter counts are a useful control, but the LSTM and Transformer have different computation graphs, gradient conditioning, and effective depth. Equal size doesn't make them equally easy to train.

My working hypothesis

For fixed-length parity, your Transformer is probably learning a global representation that makes the classification task easier to optimize, while your LSTM is struggling to establish a stable recurrent parity representation.

The most revealing detail would be **whether your Transformer still converges quickly when trained on sequences of varying lengths, say 8–32, rather than one fixed length**.

If it does, that makes the result substantially more interesting, because a simple fixed-length count-to-parity mapping becomes a less complete explanation.

my validation accuracy isn't improving very much in the eng-por problem over the lstm when i use the transformer, anything common I should look for?

Yes. **English → Portuguese translation is a much different problem from parity**, and there are several common reasons a Transformer might converge faster without producing much better validation accuracy than an LSTM. C-FR-4

My first suspects would be **masking, positional encoding, learning-rate scheduling, and dataset size**.

Unlike parity, translation rewards learning local word order, grammatical structure, long-range dependencies, and alignment between two languages. An LSTM with attention already has some useful inductive biases for those tasks.

1. Check these issues first

Potential issue	Typical symptom	What to investigate
Incorrect decoder masking	Training loss improves, translation quality stays poor	Causal target mask
Missing positional encoding	Awkward word order or poor grammar	Source and target positional information
Incorrect padding masks	Repetitions, early EOS, unstable translations	Source, target and cross-attention masking
No learning-rate warmup	Slow convergence or inconsistent results	Transformer-specific LR schedule
Too little training data	Training improves more than validation	Dropout, model capacity, regularization
Inference/training mismatch	Low teacher-forced loss, poor generated translations	Autoregressive decoding
Incorrect evaluation	Little improvement in token accuracy despite better translations	BLEU/chrF, sequence-level metrics

A useful distinction: if your Transformer and LSTM have similar validation *loss*, that suggests something different from similar validation *accuracy*. Translation accuracy can be misleading because a prediction can be linguistically acceptable without matching the reference token exactly.

2. Verify the decoder training setup

This is the first thing I'd inspect in your actual PyTorch code.

For a target sequence:

```
<BOS> eu gosto de aprender português <EOS>
```

The Transformer should see:

```
Decoder input:  <BOS> eu gosto de aprender português
Expected output: eu gosto de aprender português <EOS>
```

The decoder inputs and prediction targets must be shifted by one token.

Here's the important part of a correctly configured training step using `nn.Transformer` with `batch_first=True`:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

# src: [batch, src_len]
# tgt: [batch, tgt_len], including BOS and EOS
# src_pad_id / tgt_pad_id: padding IDs

tgt_in = tgt[:, :-1]
tgt_out = tgt[:, 1:]

src_pad = src.eq(src_pad_id)
tgt_pad = tgt_in.eq(tgt_pad_id)

# Prevent decoder from seeing future target tokens
T = tgt_in.size(1)
causal_mask = nn.Transformer.generate_square_subsequent_mask(
    T, device=tgt.device
)

# These should include embedding + positional encoding
src_emb = encode_source_embeddings(src)
tgt_emb = encode_target_embeddings(tgt_in)

hidden = transformer(
    src_emb,
    tgt_emb,
    tgt_mask=causal_mask,
    src_key_padding_mask=src_pad,
    tgt_key_padding_mask=tgt_pad,
    memory_key_padding_mask=src_pad
)

logits = output_projection(hidden)

loss = F.cross_entropy(
    logits.reshape(-1, logits.size(-1)),
    tgt_out.reshape(-1),
```

```
    ignore_index=tgt_pad_id
)
```

`encode_source_embeddings`, `encode_target_embeddings`, `transformer`, and `output_projection` are placeholders for your model's modules.

Two particularly easy mistakes:

- `nn.Transformer` **doesn't add positional encodings for you.** You must add them to both source and target embeddings.
- In PyTorch's `nn.Transformer`, a boolean attention mask value of `True` means **blocked**, not allowed. This differs from the boolean convention in `scaled_dot_product_attention`.

3. Check the optimizer before increasing model size

The original Transformer used a learning-rate schedule with warmup followed by inverse-square-root decay:

$$LR(t) \propto d_{\text{model}}^{-1/2} \min(t^{-1/2}, t \cdot w^{-3/2})$$

where w is the warmup duration. It also used dropout and label smoothing.

For a relatively small English–Portuguese dataset, I'd experiment with:

Hyperparameter	Starting experiment
Optimizer	AdamW
Peak learning rate	3×10^{-4}
Warmup	5–10% of total training steps
Dropout	0.1–0.2
Label smoothing	0.1
Gradient clipping	1.0
Vocabulary	Subword/BPE, e.g. 8k–16k tokens

These are starting points, not universal optimal values. In particular, if the model has only a few thousand total optimizer steps, copying a 4,000-step warmup from the

original Transformer may be counterproductive.

4. Your dataset size could explain much of the result

An LSTM with attention can be surprisingly competitive on small translation datasets.

The LSTM processes tokens sequentially, giving it an architectural bias toward local ordering and nearby linguistic relationships.

The Transformer must learn much of that structure from examples. Its advantage in parallel training and long-range access doesn't guarantee higher validation accuracy with limited parallel data. Low-resource machine translation remains particularly sensitive to dataset size and quality.

I would also verify that you're not using a word-level vocabulary with a large number of rare Portuguese word forms. Subword tokenization often helps with Portuguese morphology and previously unseen word variants.

5. Use the training/validation gap to narrow down the problem

Which pattern best matches your results?

- (●) Training accuracy rises, validation plateaus
- ( ) Both training and validation accuracy stay low
- ( ) Both improve, but Transformer ≈ LSTM
- ( ) Validation loss improves, but generated translations don't

Likely generalization problem. Check dataset size, rare-word coverage, duplication and train/validation leakage. Try increased dropout, smaller model width, subword tokenization, and early stopping. Training longer alone probably won't help.

6. The experiment I'd prioritize

Measure validation quality separately for short, medium, and long English sentences. Specifically, compare the LSTM and Transformer across sentence-length buckets.

If the Transformer performs similarly on short sentences but better on longer ones, that would support a benefit from its architecture. If both perform similarly

everywhere, data quality, optimization, or tokenization may be the limiting factor instead.

Also compare **chrF or BLEU** on autoregressively generated translations. Token-level accuracy is not a complete measure of translation quality because valid Portuguese translations can differ from the reference.

One other consideration from your earlier parameter-count experiment: matching parameter counts is a reasonable control, but don't assume the same number of layers, optimizer settings, or training epochs is optimal for both models.

Two things I'd be particularly interested in:

1. Approximately how many English–Portuguese sentence pairs are you training on — 10k, 100k, or millions?
2. Are you seeing the Transformer reach much higher *training* accuracy than the LSTM while validation stays similar, or are both training accuracies similar as well?

Those two details would help distinguish a generalization ceiling from a Transformer implementation or optimization issue.

i have 2 tokenization techniques, one is character-level while the other is word level, how does that affect the transformer's parameter budget compared to an lstm, and should i adjust the number of heads, d_model, or anything else? latent_size maybe?

Tokenization can dramatically change the parameter budget of both a Transformer and an LSTM. In English → Portuguese translation, the difference between character-level and word-level tokenization can actually be larger than the difference between the two architectures.

The key distinction is: C-CE-5 Continues a bit on next page

- **Word-level tokenization:** Large vocabulary, short sequences. More parameters are spent on embeddings and the output classifier.

- **Character-level tokenization:** Small vocabulary, long sequences. Fewer embedding parameters, but substantially more computation is required to process each sentence.

For your experiments, I'd generally adjust `d_model`, `dim_feedforward`, **and the number of layers**, while treating the number of attention heads as a separate architectural hyperparameter.

1. How tokenization changes the parameter budget

Suppose your translation model has:

- V_s: English vocabulary size
- V_t: Portuguese vocabulary size
- d: Transformer model dimension

A typical encoder-decoder Transformer has three vocabulary-dependent components:

Component	Parameters
Source token embedding	$V_s d$
Target token embedding	$V_t d$
Target vocabulary output projection	$V_t d + V_t$

Therefore, without weight tying:

$$\boxed{P_{\text{vocab}} = (V_s + 2V_t)d + V_t}$$

This is in addition to the attention and feed-forward parameters.

Notice that the Portuguese vocabulary appears twice: once in the decoder embedding and once in the final output projection.

If you tie the target embedding and output projection weights, you can save $V_t d$ parameters. That is often worth considering, provided the embedding and output dimensions match.

2. Concrete comparison: Transformer versus LSTM

Let's compare two translation models with the same basic width and number of encoder/decoder layers.

Assume:

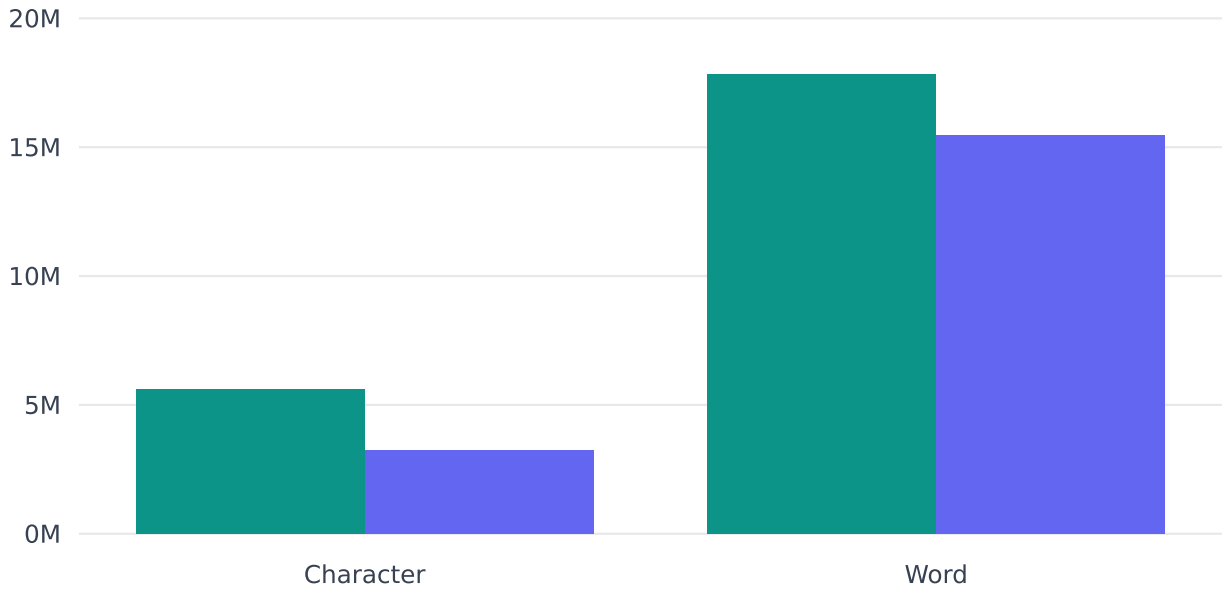
- Transformer: `d_model=256`, `dim_feedforward=1024`, 3 encoder layers, 3 decoder layers.
- LSTM: `hidden_size=256`, `embedding_dim=256`, 3 encoder layers, 3 decoder layers.
- Both use independent source and target embeddings plus a target output projection.

Parameter counts with different vocabularies

Model	Character-level (120 tokens/language)	Word-level (16k tokens/language)
Transformer	5.62M	17.83M
LSTM	3.25M	15.46M

Illustrative counts. The LSTM is unidirectional and has no additional attention module; adding encoder-decoder attention or changing the decoder readout will increase its count. Neither model uses weight tying or learned positional embeddings.

Total parameters in millions for fixed-width Transformer and basic LSTM models with character- and word-level tokenization.



At word level, the embeddings and output projection represent nearly 69% of the Transformer's total parameters in this example.

At character level, they represent only about 1.6%.

That's a major architectural difference: **word-level Transformers can spend most of their budget learning vocabulary representations, while character-level Transformers devote almost all their parameters to attention and feed-forward computation.**

The underlying encoder/decoder layers have not changed at all.

For a conventional PyTorch LSTM with input width e and hidden width h, a single recurrent layer contributes:

$$P_{\text{LSTM layer}} = 4h(e + h) + 8h$$

This assumes the default two bias vectors and no projection. Higher stacked layers use the preceding layer's output width as their input width.

3. Which Transformer hyperparameters should you actually adjust?

`d_model` **— highest-impact width parameter**

It affects token embeddings, attention matrices, feed-forward layers, and the output projection. Its effect is partly quadratic and partly linear with vocabulary size.

For a modest translation dataset, I would start with 256 and test 128 and 384.

`nhead` **— not a parameter-budget knob**

With standard PyTorch multi-head attention, changing from 4 to 8 heads does not change the projection parameter count if `d_model` is fixed.

Each head gets `d_model // nhead` dimensions. For `d_model=256`, 4 heads give 64 dimensions per head, while 8 heads give 32. I'd start with 4.

`dim_feedforward` **— useful for fine-tuning budget**

The feed-forward weights contribute $2df$ per encoder or decoder layer.

Try `2 * d_model` or `4 * d_model`. Going from 1024 to 512 with `d_model=256` saves substantial parameters without modifying the embedding dimension.

`latent_size` **— depends on your implementation**

`nn.Transformer` has no standard `latent_size` argument.

If you're using `latent_size` as the LSTM's hidden-state size, then `d_model` is its closest Transformer analogue. If it's a separately defined embedding or bottleneck size, that mapping doesn't necessarily hold.

Keep `embedding_dim = d_model` initially unless you're deliberately adding projection layers.

4. The overlooked issue: character sequences are much longer

A short translation example illustrates this.

Portuguese sentence

Eu gosto de aprender português.

Word-level	Character-level
5	**32**
word tokens (ignoring punctuation)	characters including spaces and punctuation

For dense self-attention, the attention-score computation grows approximately as:

D-DJ-6

$$O(T^2 d)$$

If character tokenization increases sequence length by a factor of 5, the quadratic attention component can require roughly 25 times as much computation, at unchanged width. Cross-attention similarly scales with source length times target length.

An LSTM's recurrent computation grows linearly with the number of time steps, although those steps have sequential dependencies.

This matters for your experiment: **matching total parameters does not match training compute, memory, or number of decoding steps.**

With characters, you may need lower `d_model`, shorter batches, or more efficient attention implementations to maintain reasonable throughput. Conversely, character-level training provides many more target-token predictions per sentence.

5. An interactive parameter-budget calculator

Use this to compare the choices without rebuilding your model for every configuration.

Transformer budget explorer

Encoder–decoder

Source vocabulary

16000

Target vocabulary

16000

`d_model`

256

Attention heads

4 heads

FFN width

4 × d_model

Encoder layers

3

Decoder layers

3

Head dimension

64

Tie target embedding and output weights

TOTAL TRAINABLE PARAMETERS

17,834,624

17.83 million parameters

Source + target embeddings	8,192,000 (45.9%)
Output projection	4,112,000 (23.1%)
Encoder	2,369,280 (13.3%)
Decoder	3,161,344 (17.7%)

Assumes standard `nn.Transformer` biases and final LayerNorms, independent source and target embeddings, output bias, and no learned positional embeddings. Tying reuses the target embedding matrix.

Copy configuration

6. My recommended experiment for your English–Portuguese setup

I wouldn't change `nhead` just to achieve a matching parameter count. Instead, I'd run a controlled comparison:

Setting	Character-level	Word-level
`d_model`	256	256
`nhead`	4	4
`dim_feedforward`	1024	1024
Encoder layers	3	3
Decoder layers	3	3
Dropout	0.1–0.2	0.1–0.2
Positional encoding	Required	Required
Padding masks	Required	Required

Then measure actual parameters and adjust `d_model` or FFN width to approach your previous LSTM's budget.

For translation quality, I'd also consider **subword tokenization (BPE or SentencePiece)** as a third experiment. A vocabulary of roughly 4k–8k subword units per language is a reasonable small-data starting point. It avoids the enormous vocabulary of word-level models without creating sequences as long as character-level tokenization.

One important experimental detail

If your LSTM has `hidden_size=512` but its embeddings are only 128-dimensional, **matching its hidden size to `d_model=512` isn't the same as matching its parameter count**. You need to account for the embedding sizes, all recurrent/attention layers, and the output projection.

For actual PyTorch models, always verify with:

```
def count_parameters(model):
    return sum(
        p.numel()
```

```
        for p in model.parameters()
        if p.requires_grad
    )

print("LSTM:", count_parameters(lstm_model))
print("Transformer:", count_parameters(transformer_model))
```

I'd be particularly interested in your **LSTM's `hidden_size` (or `latent_size`), `embedding_dim`, and number of layers**, plus the source and target vocabulary sizes for each tokenizer. With those, we can calculate the Transformer dimensions needed to match the LSTM budget and isolate architecture effects from tokenization effects.



why exactly is the transformer so much better than the lstm at both tasks? even at the same parameter count?



The key is that **equal parameter count does not mean equal computational power, equal optimization difficulty, or equal access to information.**

Your Transformer may be outperforming the LSTM because its architecture makes the particular computations needed for parity and translation easier to *learn*, even though both models theoretically have enough capacity to solve the tasks.

I think there are four major mechanisms behind your results:

1. **Shorter gradient paths:** The Transformer can learn relationships between distant tokens without backpropagating through every intermediate time step.
2. **Direct access to information:** Self-attention and cross-attention let the model retrieve relevant token representations instead of repeatedly compressing information into a recurrent state.
3. **Easier optimization:** Residual connections, LayerNorm, and parallel token representations can make the Transformer's loss landscape easier to navigate.
4. **Different use of parameters:** An LSTM repeatedly applies its recurrent weights, while a Transformer uses attention and feed-forward transformations to construct global representations.

But the explanation differs significantly between your two experiments, and there's an important distinction between **faster training convergence and better validation generalization**.

1. The biggest difference is computational depth

Let's say your input has 100 tokens.

LSTM

An LSTM processes them through a sequence of state updates:

$$h_t = \text{LSTM}(x_t, h_{t-1}, c_{t-1})$$

The influence of x_1 on h_{100} travels through 99 recurrent transitions.

The gradient contains a chain of Jacobian products:

$$\frac{\partial h_{100}}{\partial h_1} = \prod_{t=2}^{100} \frac{\partial h_t}{\partial h_{t-1}}$$

The LSTM's cell state and gates help preserve gradients, but they don't eliminate the challenge of learning useful recurrent dynamics.

Transformer

Self-attention computes:

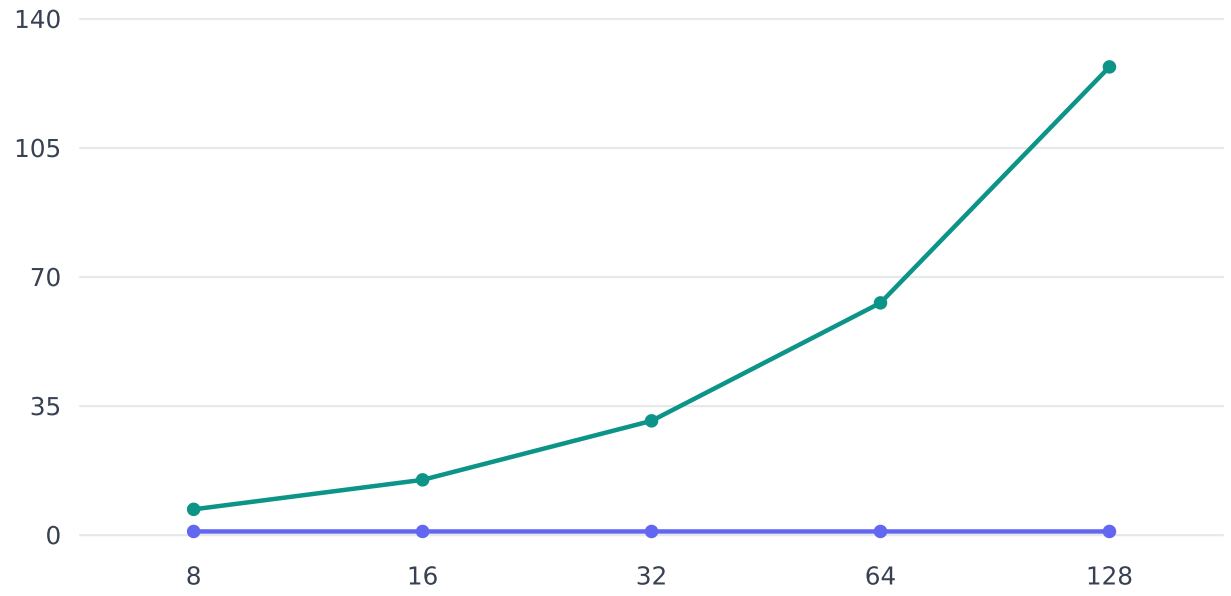
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

A representation at position 100 can depend directly on the representation at position 1 within a single attention layer, assuming the attention mask permits it.

The dependency path is therefore much shorter, and gradients also benefit from residual connections.

Dependency path as sequence length increases

Schematic shortest dependency path in recurrent and globally attending models, not measured effective gradient strength.



Illustrative dependency-path comparison; it does not imply that a single attention layer can compute every sequence function.

For your character-level English–Portuguese model, this difference becomes especially important because character sequences are much longer than word sequences.

The architecture with fewer sequential dependencies may be substantially easier to optimize, regardless of parameter count.

2. Why this helps translation

Suppose the model is translating:

> *The book that my friends bought yesterday is interesting.*

A correct translation must preserve relationships between words that may be far apart.

In a basic LSTM encoder-decoder without attention, information about the English sentence must be compressed into its recurrent representations, potentially a single final encoder state.

The Transformer decoder instead uses cross-attention:

$$\mathrm{CrossAttention}(Q_{\text{target}}, K_{\text{source}}, V_{\text{source}})$$

At every output position, it can access all encoder token representations and learn which source positions are relevant.

That creates a more direct mechanism for translating words, resolving dependencies, and retrieving source information.

Crucial caveat: an LSTM with Bahdanau or Luong attention already has access to encoder states. If your LSTM uses attention, the Transformer isn't winning simply because it can retrieve source tokens. Its additional advantages include contextualized encoder representations, decoder self-attention, shorter paths between positions, and parallelization.

These distinctions are consistent with the original Attention Is All You Need experiments, which compared Transformer machine translation against recurrent approaches, including attention-based ones.

3. Why this can help parity — but for a different reason

This is where I think your experiments become particularly revealing.

For parity,

$$y = \left(\sum_i x_i \right) \bmod 2$$

An LSTM can represent the correct algorithm using just a running binary state.

A Transformer might instead learn something like:

$$x_1, \ldots, x_T \xrightarrow{\text{attention}} \text{aggregate number of ones} \xrightarrow{\text{MLP}} \text{even/odd}$$

If the training lengths are fixed or restricted, the model only needs to distinguish a finite set of possible counts.

This may provide an effective solution to your training distribution without learning an XOR state machine.

However, parity is not intrinsically easier for Transformers. Research on Transformer sensitivity shows that parity is particularly difficult to learn and to generalize across lengths. Some studies actually find advantages for recurrent networks on parity under certain conditions.

Therefore, I would interpret your faster parity convergence as evidence that **your specific Transformer configuration has found an easier optimization route on**

your data distribution, not evidence that attention is universally better at XOR.

4. Why matching parameters doesn't create a fair computational comparison

This is an important distinction in your experiment.

Suppose both models have 5 million trainable parameters.

LSTM	Transformer
5M	**5M**
Parameters	Parameters
The same recurrent transformation is applied sequentially at every time step. Activation dependencies have to be resolved in order.	Attention computes token interactions while the feed-forward layers operate at every position. Many computations can run in parallel.

They don't have the same number of operations.

Approximately, for a sequence length T and hidden width d:

$$\begin{aligned} \text{LSTM:} \quad & O(Td^2) \\ \text{Transformer layer:} \quad & O(Td^2 + T^2 d) \end{aligned}$$

These omit some constants and assume comparable input/hidden widths.

The Transformer can perform more arithmetic with the same number of parameters, especially on longer sequences. Yet much of that arithmetic is parallelizable, which often makes it faster on GPUs.

Equal parameters therefore control **model storage capacity**, not computational effort.

5. Why your validation results matter

You mentioned earlier that the English–Portuguese Transformer converges faster but doesn't improve validation accuracy very much.

That's an important result.

It suggests that the Transformer may be substantially better at optimizing the training objective without being proportionally better at generalizing to unseen translations.

Consider these possible outcomes:

Observation	Likely interpretation
Transformer train loss falls much faster; validation is similar	Optimization advantage, possible generalization ceiling
Both reach similar train loss, Transformer needs fewer steps	Optimization advantage
Transformer improves validation loss and chrF/BLEU	Evidence of better translation generalization
Transformer beats LSTM only on character tokenization	Sequence-length effects may be important
Transformer fits fixed-length parity but fails longer sequences	Possible counting shortcut rather than general XOR

6. The experiments I'd run to find the actual cause

I'd prioritize these four ablations:

1. **Give the LSTM attention.** If you're currently using a plain encoder-decoder LSTM, this is the most informative change. It isolates the benefit of accessing source representations from the benefit of removing recurrence.

2. **Compare word-level and character-level performance at matched training FLOPs.** Character tokenization greatly increases sequence length, which changes both models' computational workloads.

3. **Match optimization budgets, not just parameter counts.** Compare steps to a target validation loss, wall-clock time, total training tokens, and ideally estimated FLOPs.

4. **Test parity at unseen lengths.** If the Transformer is faster on training lengths but the LSTM generalizes further, that indicates different learned algorithms rather than an unqualified architectural advantage.

My interpretation of your results

Parity: Your Transformer may be taking advantage of a parallel, fixed-length computation that is easier to optimize than learning recurrent XOR updates.

English–Portuguese: The Transformer likely benefits from direct token interactions and more efficient gradient propagation, particularly with character-level sequences. If

validation quality barely improves, the remaining bottleneck may be data, tokenization, decoding, or generalization.

The one architectural detail I'd most like to know is **whether your LSTM translation model uses attention or just passes the final encoder hidden/cell states to the decoder**.

That distinction could explain a substantial part of the Transformer–LSTM difference you're observing.